

南京信息工程大学 实验（实习）报告

实验（实习）名称 压缩编码 日期 2026.4.21 得分

指导教师 崔琦 班级 24 奇安信 1 班 姓名 孙涛 学号 202483950014

一、实验目的

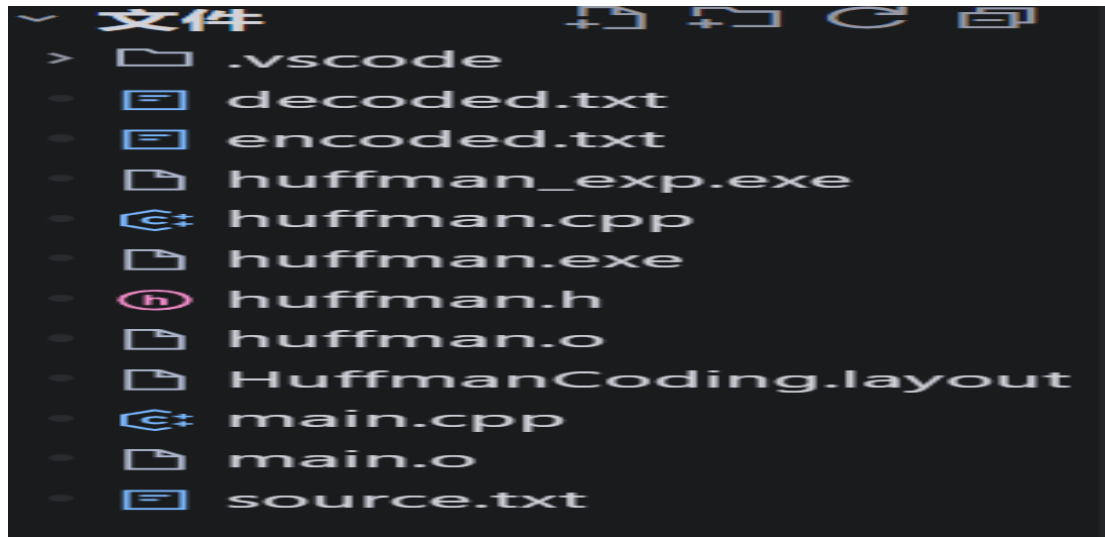
使用 **huffman** 编码对文本文件进行压缩并生成压缩文件，将压缩文件再解压缩与源文件进行比较，理解熵与平均码长的关系，以及如何计算编码效率，验证 **huffman** 编码作为无失真压缩算法的有效性。

二、实验原理

1. 统计频率（权重）：对源文本文件进行扫描并去除有误和低频的字符，建立字符和其出现频率的对应关系。
2. 创建 **Huffman** 树：贪心算法每次取出字符中最小和次小的两个节点并合并得到根节点，直到所有节点被取用完。
3. 得到 **Huffman** 编码：从根节点出发找子节点，往左记为 0 往右记为 1，得到的每个叶子节点对应的 **Huffman** 编码
4. 数据压缩：将 01 构成的长序列每 8 位打包为一个字节存入二进制文件。
5. 解压缩：读取压缩后的文件以重建树，通过逐位匹配检验是否无失真。

三、实验内容

本次实验使用 c 语言（基于 Trae 开发环境）编写了如图结构的项目以实现本次实验的需求。实验的核心内容顺序如下：编写菜单实现互动可视化，将原本的实验一 `little_prince_sample.docx` 另存为 `source.txt` 以便使用，读取 `source.txt` 并初始化生成 **huffman** 树，查看 **huffman** 编码表（即每个字符对应的 **huffman** 编码），编码文件并压缩存入二进制文件，对二进制文件进行解压，通过公式计算文件的信息熵，平均码长和编码效率，计算实际压缩率，通过比对解压文件与源文件看是否无失真。



四、实验代码和结果截图

本实验所用核心代码依据调用顺序粘贴如下：

// 统计字符频率并返回总字符数

```
long CountFrequency(const char *filename, int *freq_array) {  
    FILE *fp = fopen(filename, "r");  
    if (!fp) return -1;  
    for (int i = 0; i < ASCII_SIZE; i++) freq_array[i] = 0; //清空字符频率数组  
    long count = 0;  
    int ch;  
    while ((ch = fgetc(fp)) != EOF) {  
        if (ch >= 0 && ch < ASCII_SIZE) {  
            // 保留英文字符（大小写）、空格、换行符和常见标点符号  
            if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z') ||  
                ch == ' ' || ch == '\n' || ch == '.' || ch == ',' ||  
                ch == '!' || ch == '?' || ch == ';' || ch == ':' ||  
                ch == '"' || ch == '\'' || ch == '(' || ch == ')' ||  
                ch == '-' || ch == '_' || ch == '=' || ch == '+' ||  
                ch == '[' || ch == ']' || ch == '{' || ch == '}' ||  
                ch == '|' || ch == '\\' || ch == '>' || ch == ':' ||  
                ch == '<' || ch == '>' || ch == '/' || ch == '?') {  
                freq_array[ch]++; //统计字符频率  
            }  
        }  
        count++;  
    }  
    return count;  
}
```

```

        count++; //统计总字符数
    }
}
}
fclose(fp);
return count;
}

//功能: 构建 Huffman 树
void CreateHuffmanTree(HuffmanTree &HT, int *weights, int n) {
    if (n <= 1) return;

    int m = 2 * n - 1; //原始有 n 个节点, 每次合并减少一个节点, 共需要 2n-1
    个节点

    HT = (HuffmanTree)malloc((m + 1) * sizeof(HTNode)); //0 号位置不用

    for (int i = 1; i <= n; ++i) {
        HT[i].weight = weights[i - 1];
        HT[i].parent = 0;
        HT[i].lchild = 0;
        HT[i].rchild = 0;
    } //将原始的 n 个节点写入树中

    for (int i = n + 1; i <= m; ++i) {
        HT[i].weight = 0;
        HT[i].parent = 0;
        HT[i].lchild = 0;
        HT[i].rchild = 0;
    }
}

```

}//将剩余的节点写入，权值暂时为 0

int s1, s2;

for (int i = n + 1; i <= m; ++i) {

Select(HT, i - 1, s1, s2); //以 i-1 作为 end，也就是说每次只处理 1 到 i-1 的节点

//找到权值最小的两个节点，将它们的权值相加作为新节点的权值，并将新节点作为它们的父节点

HT[s1].parent = i;

HT[s2].parent = i;

HT[i].lchild = s1;

HT[i].rchild = s2;

HT[i].weight = HT[s1].weight + HT[s2].weight;

}

}//成功构建 Huffman 树

//功能：从叶子节点开始，为每个字符生成 Huffman 编码

void GenerateHuffmanCodes(HuffmanTree HT, HuffmanCode &HC, char

*chars, int n) {

HC.chars = (char *)malloc((n + 1) * sizeof(char));

//只有原始的 n 个节点需要 huffman 编码，0 号位置不用，所以分配 n+1 的空间

//分配空间来存储实际的字符

HC.HCPointer = (char **)malloc((n + 1) * sizeof(char*));

//分配空间来存储每个字符对应的 Huffman 编码，每个编码都是由 01 组成的，所以用二级指针

```

// 临时空间，用来存储当前生成的编码串
char *cd = (char *)malloc(n * sizeof(char));
cd[n - 1] = '\0'; // 结束符

for (int i = 1; i <= n; ++i) {
    HC.chars[i] = chars[i - 1]; // 存储字符信息
    int start = n - 1;          // start 指向编码的起始位置
    int c = i;                  // c 为当前节点
    int f = HT[i].parent;       // f 为父节点

    while (f != 0) {
        if (HT[f].lchild == c) {
            cd[--start] = '0'; // 左子树为 '0'
        } else {
            cd[--start] = '1'; // 右子树为 '1'
        }
        c = f;
        f = HT[f].parent; // 从叶子节点向上，直到根节点
    } // 此时 cd 中存储的就是 chars[i-1] 对应的 huffman 编码

    // 为第 i 个字符分配相应的空间，去掉多余部分
    HC.HCPointer[i] = (char *)malloc((n - start) * sizeof(char));
    strcpy(HC.HCPointer[i], &cd[start]);
}

free(cd); // 释放临时空间
}

void EncodeSourceFile(const char *src_file, const char *dst_file,
HuffmanCode HC, int *char_map, int n) {

```

FILE *fin = fopen(src_file, "r");//打开源文件

FILE *fout = fopen(dst_file, "wb");// 以二进制模式打开目标文件，用于写入编码后的数据

```
if (!fin || !fout) {  
    printf("错误: 无法打开文件。 \n");  
    return;  
}
```

// 写入字符数量

fwrite(&n, sizeof(int), 1, fout);

// 写入字符-频率表

int freq[ASCII_SIZE] = {0};//字符频率数组，用于存储每个字符的出现次数

CountFrequency(src_file, freq);//统计字符频率

```
for (int i = 1; i <= n; i++) {  
    char c = HC.chars[i];  
    int frequency = freq[(unsigned char)c];  
    fwrite(&c, sizeof(char), 1, fout);//写入字符  
    fwrite(&frequency, sizeof(int), 1, fout);//写入字符频率  
}
```

unsigned char buffer = 0;//用于存储编码后的二进制数据

int bit_count = 0;//用于记录当前缓冲区已使用的位数

int ch;

while ((ch = fgetc(fin)) != EOF) {

int index = char_map[ch];

if (index != 0) {

char *code = HC.HCPointer[index];

for (int i = 0; code[i] != '\0'; i++) {

```

        buffer <<= 1; //将缓冲区左移一位，为下一位做准备
        if (code[i] == '1') {
            buffer |= 1; //如果当前位为 1，将缓冲区的最低位设为 1
        }
        bit_count++; //记录当前位数
        // 当缓冲区满 8 位时，写入到目标文件
        if (bit_count == 8) {
            fwrite(&buffer, sizeof(unsigned char), 1, fout);
            buffer = 0;
            bit_count = 0; //清空缓冲区，准备写入下一位数据
        }
    }
}

// 处理最后不足 8 位的情况
if (bit_count > 0) {
    buffer <<= (8 - bit_count); //将缓冲区左移，将未使用的位设为 0
    fwrite(&buffer, sizeof(unsigned char), 1, fout); //写入未使用的位到目标
文件
}

fclose(fin);
fclose(fout);
printf(">>> 编码完成! 请在%s 中查看\n", dst_file);
}

```

实验结果测试图如下：

```
PS C:\Users\23059\Desktop\huffmancode> g++ main.cpp huffman.cpp -o huffman_exp
PS C:\Users\23059\Desktop\huffmancode> ./huffman_exp
```

```
=====
      Huffman 编码解码系统
=====
1. 初始化并生成 Huffman 树 (读取 source.txt)
2. 查看 Huffman 编码表
3. 编码文件 (source.txt -> encoded.txt)
4. 解码文件 (encoded.txt -> decoded.txt)
5. 信息论定量分析 (信源熵、平均码长、编码效率、压缩率)
6. 无失真校验 (对比 source.txt与 decoded.txt)
0. 退出系统
=====
请选择操作 [0-6]: 1
Huffman 树生成成功! (总字符数: 2209)
```

```
0. 退出系统
=====
请选择操作 [0-6]: 2

--- Huffman 编码表 ---
'\n': 11000110
' ': 111
',' : 001000
'-' : 0101110011
'.' : 001001
'A' : 110001110
'E' : 1100010100
'H' : 1100010111
'I' : 11000101100
'O' : 010111010
'T' : 11000100
'W' : 010111000
'a' : 1000
'b' : 101000
'c' : 110101
'd' : 10101
'e' : 011
'f' : 010100
'g' : 101001
'h' : 0001
'i' : 0011
'j' : 11000101101
'k' : 110001111
'l' : 11001
'm' : 010101
'n' : 1001
'o' : 0100
'p' : 110000
'q' : 010111011
'r' : 11011
's' : 0000
't' : 1011
'u' : 00101
'v' : 0101111
'w' : 110100
```



```
6. 无失真校验 (对比 source.txt与 decoded.txt)
0. 退出系统
```

```
=====
请选择操作 [0-6]: 5
```

```
=== 信息论定量分析 ===
信源熵 H(X): 4.2917 bits/符号
平均码长 L: 4.3232 bits/符号
编码效率  $\eta$ : 99.27%
```

```
=== 压缩率分析 ===
原文件大小: 2220 字节
压缩后文件大小: 1388 字节
压缩率: 62.52%
```

```
6. 无失真校验 (对比 source.txt与 decoded.txt)
0. 退出系统
```

```
=====
请选择操作 [0-6]: 6
正在进行无失真校验...
```

```
✓ 校验通过! 源文件与解压文件完全一致 (无失真)
```

补：使用 Hex editor 查看二进制文件 encoded.txt 结果如图：

Hex	Decoded Text
26 00 00 00 0A 0B 00 00 00 20 70 01 00 00 2C 1B	& P
00 00 00 2D 02 00 00 00 2E 1B 00 00 00 41 06 00	 A
00 00 45 02 00 00 00 48 03 00 00 00 49 01 00 00	. . E H I
00 4F 04 00 00 00 54 09 00 00 00 57 03 00 00 00	. O T W
61 81 00 00 00 62 22 00 00 00 63 31 00 00 00 64	a . . . b " . . . c 1 d
4B 00 00 00 65 F4 00 00 00 66 1C 00 00 00 67 22	K . . . e . . . f . . . g "
00 00 00 68 6A 00 00 00 69 71 00 00 00 6A 01 00	. . . h j . . . i q . . . j
00 00 6B 06 00 00 00 6C 5C 00 00 00 6D 20 00 00	. . k l \ m
00 6E 82 00 00 00 6F 76 00 00 00 70 22 00 00 00	. n o v . . . p "
71 04 00 00 00 72 5F 00 00 00 73 69 00 00 00 74	q r _ . . . s i t
A2 00 00 00 75 39 00 00 00 76 12 00 00 00 77 2D	. . . u 9 . . . v . . . w -
00 00 00 78 02 00 00 00 79 20 00 00 00 7A 01 00	. . . x y z
00 00 5D 4E AF D8 BD BF A4 07 8F 93 BB CB F8 6C	. .] N l
E7 57 F4 14 F9 35 EE BD 27 C7 C3 31 2E F8 6B 1B	. W . . . 5 . . . ' . . . 1 . . k . . .
D5 E5 6F 41 D3 4B DF B1 89 F1 C5 0A 0C 9F 8B BF	. . o A . K
94 5E EB C0 B2 0E C0 8E BB 29 BB EB B2 B0 B3 49	. ^ . o A . K
02 3C 4D 7D 8B FA A2 B8 44 AD D1 4F 1C 0E 69 CB	. < M } D . . O . . i . . .
FB 40 64 FC 51 7B DA DD 54 DC 9C D3 C5 FD 72 E2	. @ d . Q { . . . T r . . .
5D 78 98 75 E9 9D 62 51 87 89 AF C0 B9 CB AF 2E	] x . u . . b Q
1E C5 FB 39 5B 80 A2 58 74 53 D1 09 44 51 EF 6D	. . . 9 [. . X t S . . D Q . m . . .
87 A1 A8 9B 7E C5 AD F5 42 E6 BE 9D A6 9E D1 3E	. . . ~ . . . B >
63 74 B2 7E 2E FD 0F 26 D7 BA FB 18 BE 0A C6 73	c t . ~ . . . & s . .
EA 5B 36 04 73 53 D5 25 FD 0E C7 EB 1B 64 7D 5D	. [6 . s S . % d } P . . .
B9 AF C3 69 6F 5B F1 3D CD 9E DF D1 37 9A 93 8D	. . . i o [. = 7 . . .
8D 88 2F E4 EE F2 FE 1B 39 D5 FD 20 3E A5 D9 A1	. . / 9 > . . .
43 C5 08 5B ED 7B DA D6 27 34 93 F1 77 E0 31 EE	C . . [. { . . . ' 4 . . w . 1 . . .
BF 41 5B D8 BE 17 1B 0E 02 92 C8 FA 0A DD 4C A6	. A [. L . . .

五、实验心得

本次实验通过 huffman 编码实现了将文本文件无失真压缩与解压缩，而实验得到的编码效率 η 是 99.27% 的原因是离散性约束，huffman 编码要求码长必须为整数（比特），而信源中的单个字符的理想信息量 $-\log_2(p_i)$ 通常是小数，当且仅当所有字符的 p 都是 2 的负整数次幂时，效率才能达到 100%。本实验验证了 huffman 编码为无失真压缩且能够有效降低文本冗余。